



Optimizing data lakehouses with Starburst

HDFC Private Training

Trainer

Hitesh Kumar Sharma

Optimizing data lakehouses with Starburst

This course features instructor-led discussions, demonstrations, and hands-on exercises designed to build a working knowledge of the Starburst query engine.

Upon completion, you will gain a more thorough awareness of Starburst architecture, with a specific focus applied to best practices for data lake based schemas, including table formats and partitioning, file formats and sizes, and other optimization techniques.

Course introduction

Who, what, where, when & why

Course objectives

- Use Starburst as a single point of access for multiple data sources and federate queries across them
- Evaluate and describe how queries are executed within a Starburst cluster
- Use Hive and Iceberg table formats; construct, populate, query, and modify partitioned tables
- Employ file size/format/hierarchy strategies to improve query performance
- Understand the role of the Cost-based optimizer and read query plans to ensure optimizations are occurring as expected and to identify possible issues
- Build a data engineering pipeline with Starburst Galaxy

What to expect

- Blend of conceptual and practical
- Focused learning objectives
- Instructor demonstrations
- Plenty of hands-on labs
- Interactive discussions
 - Ask questions
 - Validate assumptions
 - Discuss use cases
 - FEEDBACK IS OXYGEN

Course agenda

1 - Data lake tables

- Separation of storage & compute
- Schema on read

2 - Data lake performance

- Limit data exchanges
- File format options
- Small files problem
- Partitioning & bucketing

3 - Apache Iceberg

- Table format architecture
- Creating tables
- Insert, delete & update

4 - Advanced Iceberg

- CDC with merge
- Schema & partition evolution
- Snapshots & compaction

5 - Parallel processing

- Divide & conquer
- Beyond single stage queries

6 - Cost-based optimization

- Overview
- Query plan analysis

7 - Data pipelines

- Definition & differentiation
- Reference architecture

Introductions

Please feel free to share as much (or as little) as you feel comfortable with

- You! What's your name?
 - Where are you location? Working remote?
 - High-level role (company and/or project)
 - Any hobby/activity you want to share?
- *Identify the email address you want for your Starburst training ID*
- Prior experience with
 - Trino (Presto) and/or Starburst (Enterprise and/or Galaxy)
 - Other open-source big data solutions such as Apache Hive or Spark
- Any expectations that appear to not be addressed in the agenda

Data lake tables

Understanding and creating

Lesson objectives

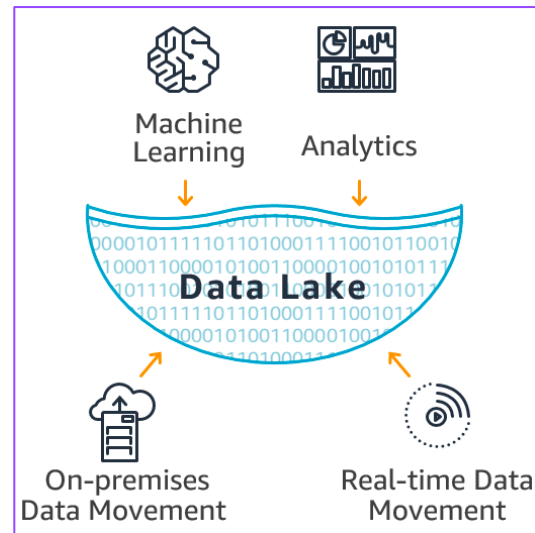
Data lake tables: Understanding and creating

1. Define the data lake.
2. Become aware that multiple file formats can be used in data lakes.
3. Explain the nature of schema on read, and compare it to schema on write in both data lakes and data warehouses.
4. Describe the role that the Hive metastore plays in data lakes.
5. Discuss the difference between external and managed tables.

What is a data lake?

"A data lake is a system or repository of data stored in its natural/raw format"

- Supports business units having TB/PB of data in different formats
- Cheap decoupled storage that can be reused for multiple purposes by multiple engines & frameworks
- Strategically aligned with Cloud Vendors (compute on-demand)
- Everyone is trying to do ML / AI projects
- New generation of Developers/Architects/Data People with skills acquired after 2006+ (Hadoop Era)



Components of a modern data lake

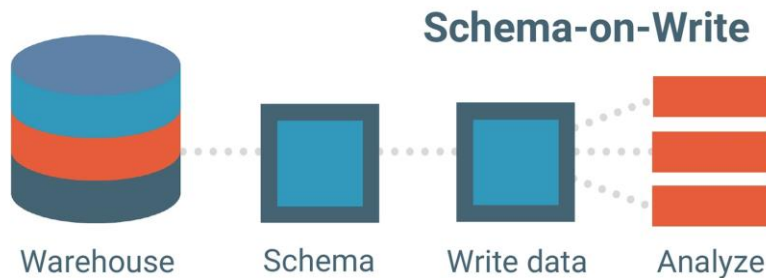


Schema considerations

Data lake vs data warehouse

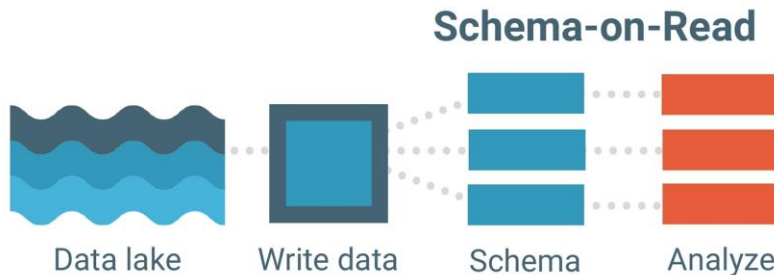
Data warehouse: Schema on Write

- Schema must be pre-defined
- Full relational data model required
- Data must be “transformed” to match schema & data model before it can be loaded into DW



Data lake: Schema on Read

- Load data in its raw form (historically immutable)
- Schema can be defined or inferred from raw data as it's read
- Schema can evolve easily as data changes
- Lacks PK/FK integrity checks



Schema on read requirements

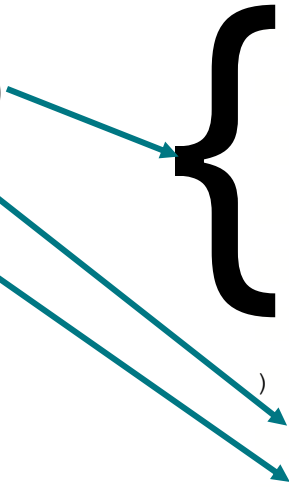
3 pieces of information are required:

1. What does it look like? (the schema)

2. Where is it? (the directory location)

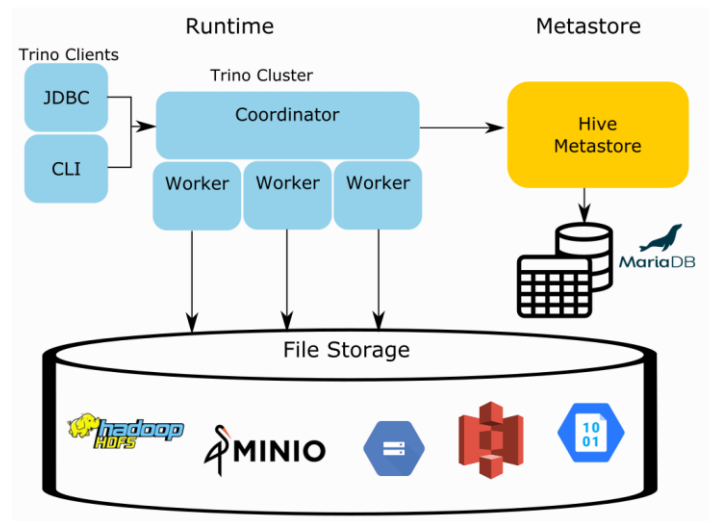
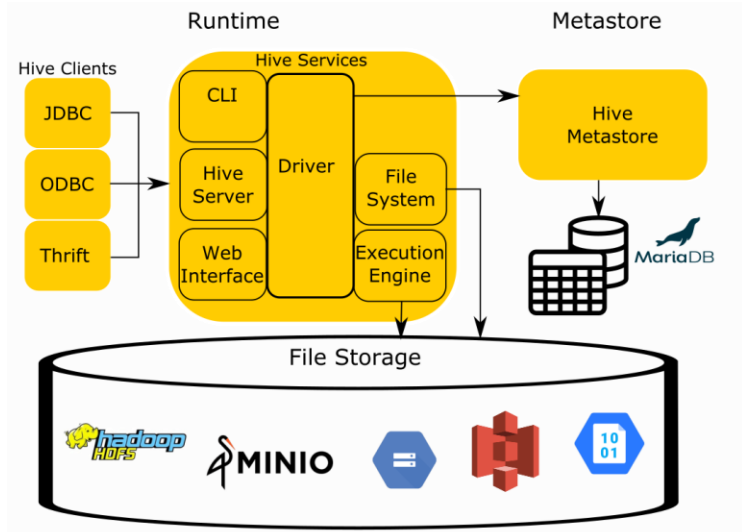
3. What will I find there? (the file type)

```
CREATE TABLE mycat.myschema.mytable (  
    event_time      TIMESTAMP,  
    ip_address      VARCHAR(15),  
    app_name        VARCHAR(25),  
    process_id      SMALLINT,  
    log_level       VARCHAR(15),  
    message_details VARCHAR(555)  
) WITH (  
    external_location = 'objStore/mytable/',  
    format = 'ORC'  
);
```



Reusing the Metastore (ONLY) from Hive

Hive vs Trino



External vs managed tables

External tables

An approach where the underlying data files are created prior to the table being created.

Identifiable by an `external_location` property in the table creation's `WITH` clause.

Characteristics

- Usually, Starburst will not write files to this data lake location
- If the table is dropped
 - The metadata is deleted
 - The underlying data files remain
- Often, leverage file formats such as CSV or JSON

Managed tables

Strategy where Starburst is creating the underlying data files as data as needed.

Normally, these tables default to being stored as a subdirectory under their schema's location.

Characteristics

- Starburst creates any necessary files to implement insert/update/delete operations
- If the table is dropped
 - The metadata is deleted
 - The underlying data files are deleted
- Often, leverage file formats such as ORC or Parquet

Instructor demonstration

Create tables (15 mins)

Hands-on exercises

Lab 1: Create a schema and
tables (20 mins)

Lab 2: Investigate Hive's special
columns (15 mins)

Lesson summary

Data lake tables: Understanding and creating

1. Data lakes store structured, semi-structured, and unstructured data and make use of traditional table formats like Hive, often using cloud object storage or other storage means.
2. Data lakes make use of multiple file formats like ORC, Parquet, and Avro.
3. Schema on read applies schemas only when data is read, whereas schema on write applies schemas when data is added to a data lake.
4. Data lakes use metastores to manage metadata, and the Hive Metastore is one of the most common implementations.
5. External tables often are populated, and accessed, by other systems, whereas underlying files for managed tables are created directly by the query engine.